

CMPSC 473 Project 3 Report

Team Members: Sivani Devarapalli(lfd5379), Yogeshvar Reddy Kallam(yvk5381)

Core Implementation

The basis of the memory manager is the utilization of the Linux `mprotect` system call along with `SIGSEGV` signal processing to emulate the hardware paging mechanisms. We start by protecting the whole virtual memory region with `PROT_NONE`, thus making it inaccessible. Any attempt by the main program to read or write to this region would raise a `SIGSEGV` signal.

We registered a custom signal handler through `sigaction` using the `SA_SIGINFO` flag, which allowed us to intercept these signals and act as the system's "Memory Management Unit" (MMU). The handler extracts the faulting address from the `siginfo_t` structure to calculate the Virtual Page Number (VPN). Most importantly, it determines whether the access was a read or a write by looking into the CPU error register (`REG_ERR`) inside the `ucontext_t` structure.

The signal handler classifies faults into two types: The first is a "True Page Fault," caused by a page not being in physical memory (`present == 0`). It then assigns a physical frame. If the memory is full, it then invokes the eviction logic to choose victim page(s) according to the active policy. If the victim page has been modified, `mod_bit` equals to 1, it sets the page as dirty and schedules it for writeback. Then, it maps the new page to the frame and updates the corresponding page table entry.

The second kind is for "Protection Fault," when a page is present but doesn't have appropriate permissions to perform the operation on it; for instance, trying to write on a Read-Only page. The above mechanism is crucial for maintaining the Reference `ref_bit` and Modified `mod_bit` required by the Third Chance algorithm.

Page Replacement Policies

The FIFO policy is the simpler of the two algorithms. It keeps a circular queue of virtual page numbers that track the order in which pages were brought into memory. When eviction needs to occur, the page at the tail of this queue the oldest page is chosen as the victim. Our implementation minimizes signal overhead by setting the permission for a page to `PROT_READ` on a read fault and only upgrading it to `PROT_READ | PROT_WRITE` upon a

write fault. This way, we will catch the first write to a page in order to set the `mod_bit` correctly and hence keep track of dirty pages as required.

The Third Chance policy is a more sophisticated variation of the Clock algorithm, mostly inspired by prior work. It treats physical frames as a circular buffer and uses a "clock hand" to search for a victim. Ideally, it seeks a page that is neither referenced nor modified ($R=0$, $M=0$). If it encounters a referenced page ($R=1$), it gives the page a "second chance" by zeroing the reference bit and advancing the hand.

This policy is unique in that it gives modified pages ($M=1$) a "third chance" to avoid the most expensive write-back operation. We use `mprotect` aggressively to strictly emulate the hardware setting of the bits. A page with `ref_bit == 0` is set to `PROT_NONE` so that any access will trap to the handler, setting the bit. A page with `ref_bit == 1` and `mod_bit == 0` is set to `PROT_READ` to trap write attempts. This careful orchestration allows software to accurately track page usage when hardware support is unavailable.

Conclusion

The solution makes heavy use of Linux-specific and architecture-specific features. Access to the write-fault bit in the `REG_ERR` register of the `ucontext_t` structure is x86_64-specific and had to be adapted in a non-straightforward manner for macOS/ARM environments during the development of this homework.

Also, `SIGSEGV` handling has significant performance overhead compared to hardware paging, and the current approach is applicable only for simulation and education purposes, not for productive memory management.

The project successfully developed and deployed a working user-space virtual memory manager. Utilizing signal handling and memory protection, we did an effective simulation of the hardware-OS interface.

The implementation shows the key trade-offs in OS design: the simplicity and low overhead of FIFO versus the improved accuracy and "fairness" of the Third Chance algorithm that keeps frequently used and dirty pages in memory at the added cost of increased complexity. The resulting library effectively demonstrated the mechanics of demand paging, address translation, and page replacement policies.